

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7 - Amrut Anand - Roll: 24011101003

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

Setup used:

- Full MNIST: 55,000 train, 5,000 validation (taken from the training set), 10,000 test.
- Adam (10^{-3}), batch size 128, 20 epochs (AEs), 30 epochs (VAE), seed 7.
- Same split and settings for every model.

3. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

4. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

5. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

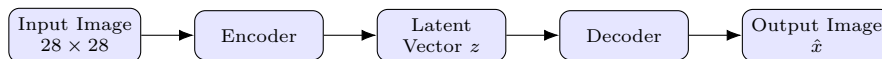
$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

6. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

Used as given (ReLU in the latent layer too). Parameters: 211,040. Training time: 39.3 s.

7. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

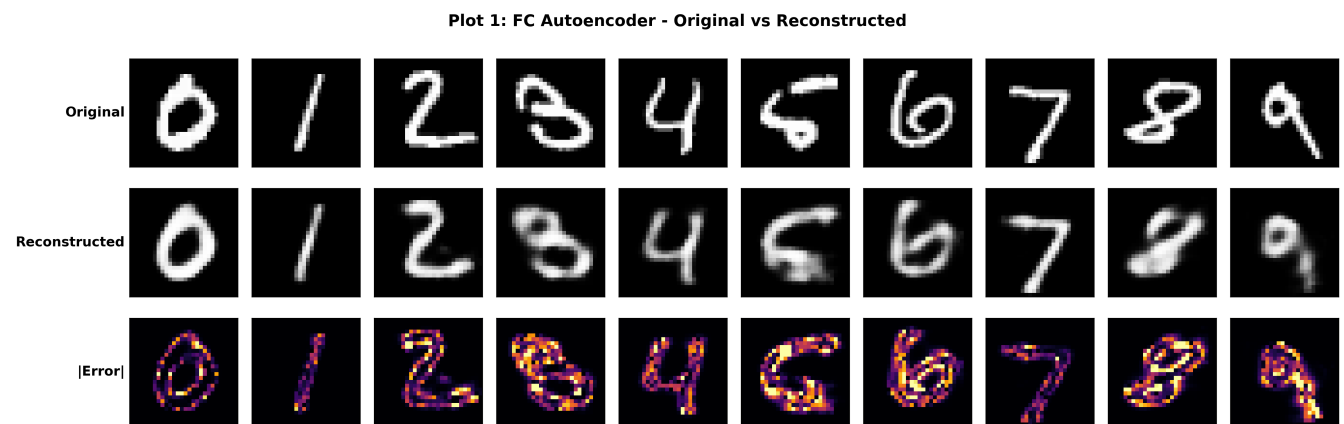


Figure 1: Plot 1. FC-AE: original, reconstruction and absolute error (one test image per digit).

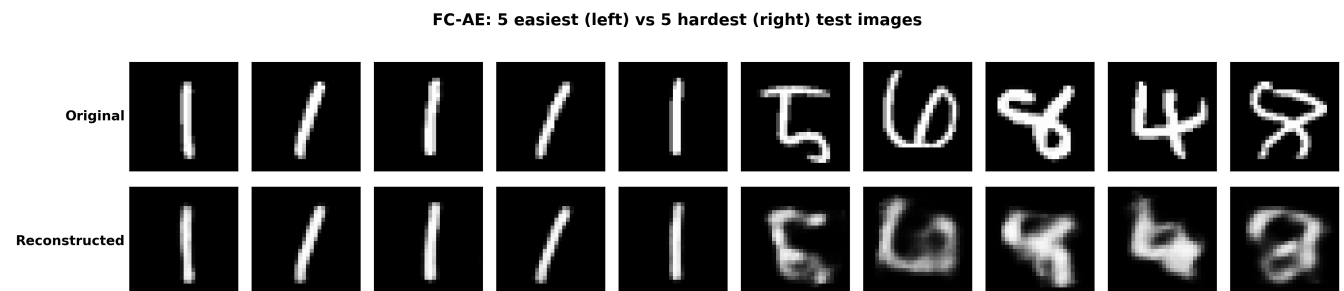


Figure 2: Five lowest-error (left) and five highest-error (right) test images.

Required inference:

Alongside the five lowest and highest-error samples, plot 1 contrast a 784-pixel original images with their 16-D FC-AE reconstructions. Complex digits (like 8s) suffers significant blurring, while basic strokes (like 1s) are preserved.

Because an severe 16-D bottleneck flatten spatial locality, this occur by forcing a network to output averged features. Consequently, the FC-AE capture global structure but fails to reconstructs high-frequency details.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

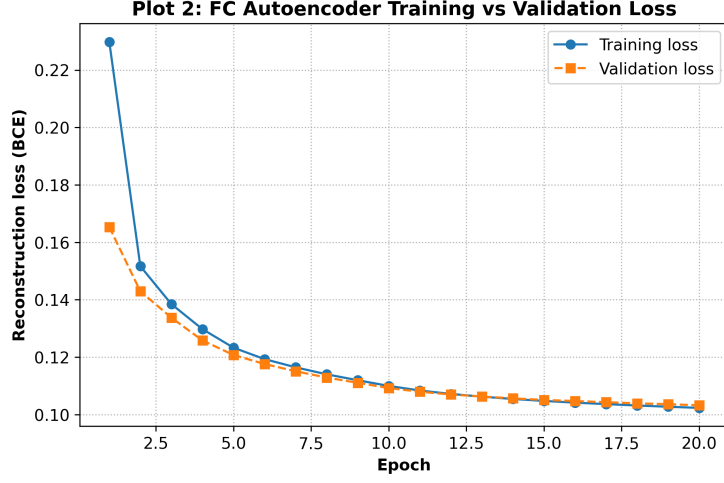


Figure 3: Plot 2. FC-AE training vs validation BCE loss.

Required inference:

Before plateauing around 0.10, a FC-AE training and validation BCE loss drop sharply from roughly 0.23 over 20 epochs. That a network rapidly extract prominent feature up to its 211,040-parameter capacity without memorizing an training sets is indicated by an heavily overlapping curves. Therefore, an model converge stably and successfully avoid overfitting.

8. Reconstruction Metrics

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

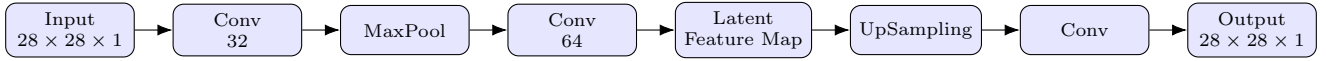
Computed per image with `skimage.metrics.structural_similarity` (data range 1), averaged over the test set.

Metric	Value
Test MSE	0.0133
Test MAE	0.0406
Mean SSIM	0.8509

9. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transpose convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```

Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
  
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

I used this implementation as written: 74,497 parameters, trained in 68.9 s. Its latent is a $7 \times 7 \times 64$ map (3,136 values), which is larger than the 784-pixel input.

10. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.0133	0.0406	0.8509	211,040
Convolutional AE	0.0012	0.0102	0.9887	74,497

Plot 3: Reconstruction Comparison

Original $\rightarrow$ FC-AE Reconstruction $\rightarrow$ CAE Reconstruction.

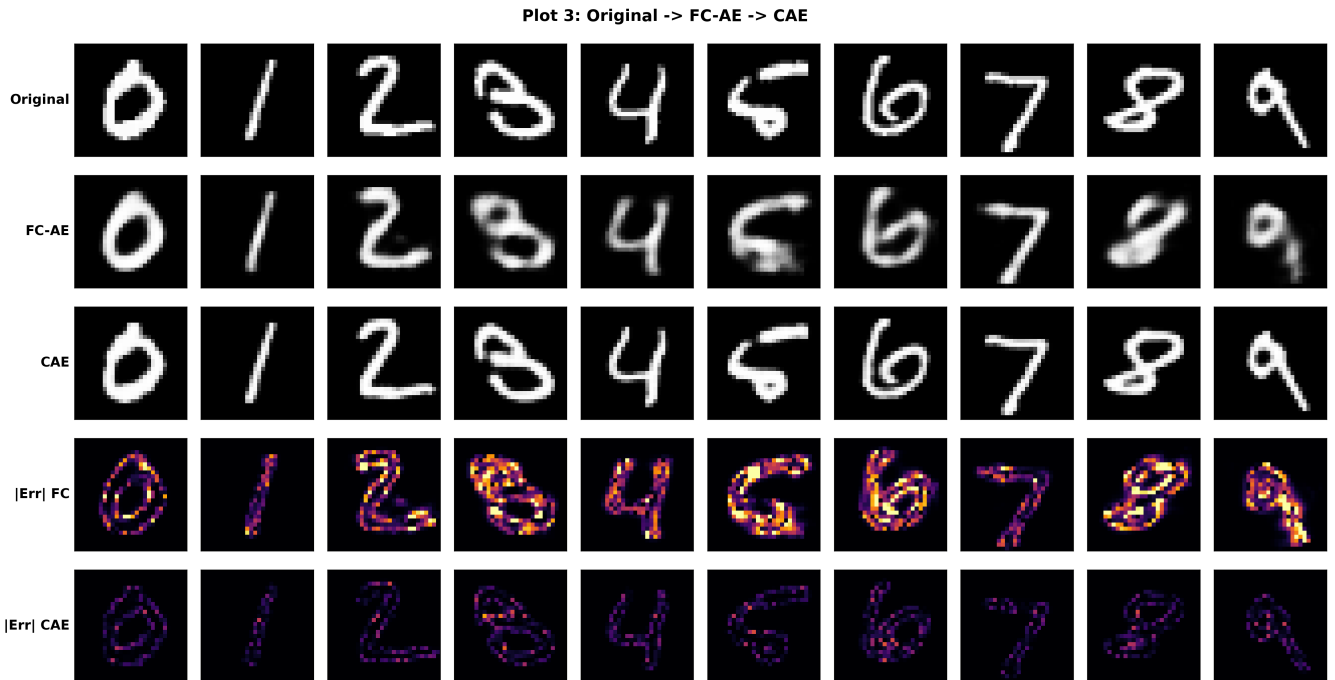


Figure 4: Plot 3. Original, FC-AE and CAE reconstructions with their error maps.

Required inference:

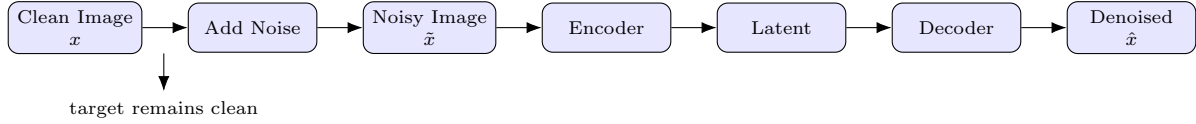
Showing that a CAE achieve an $11\times$ lower MSE (0.0012) and much higher SSIM (0.9887) than a FC-AE (0.0133 MSE, 0.8509 SSIM), plot 3 compare reconstructions. Because convolutions preserves 2D spatial locality, allowing an network to effectively captures edge and local feature, this dramatic improvment occurs. For spatial reconstruction task, we concludes that convolutional architectures is substantially superior to fully connected model.

11. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image. The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



Same architecture as the CAE. I trained two separate models: one on Gaussian noise ($\sigma = 0.2$) and one on salt-and-pepper noise ($p = 0.1$). Fresh noise was generated every epoch.

12. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

13. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

Clean | Noisy | Denoised.

Plot 4: Clean vs Noisy vs Denoised (Gaussian 0.2)

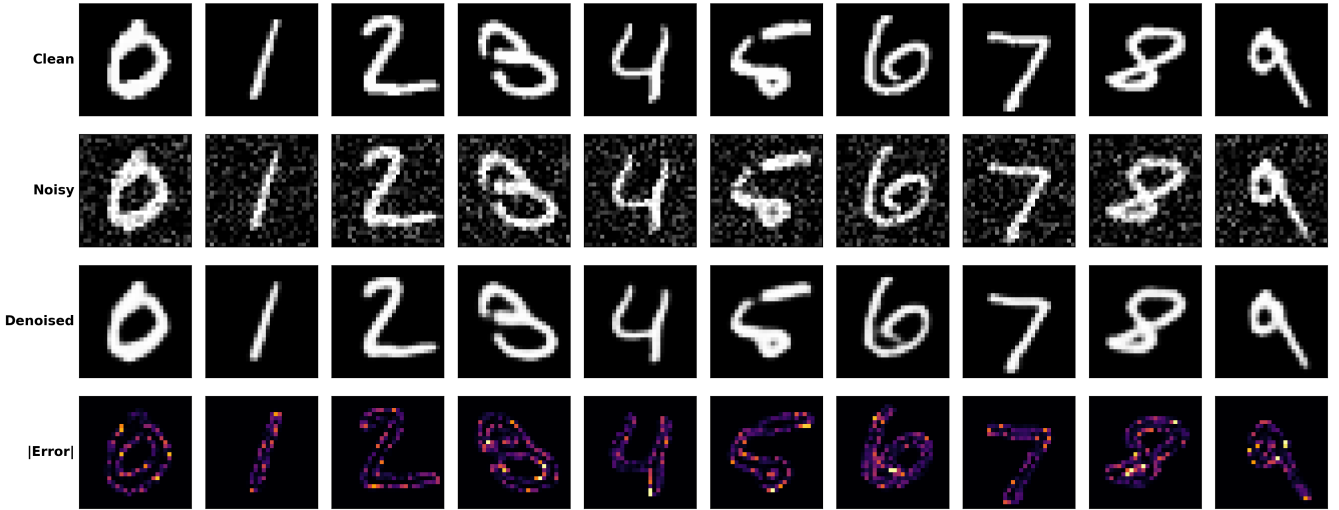


Figure 5: Plot 4(a). Gaussian noise, $\sigma = 0.2$.

Plot 4: Clean vs Noisy vs Denoised (S&P 0.1)

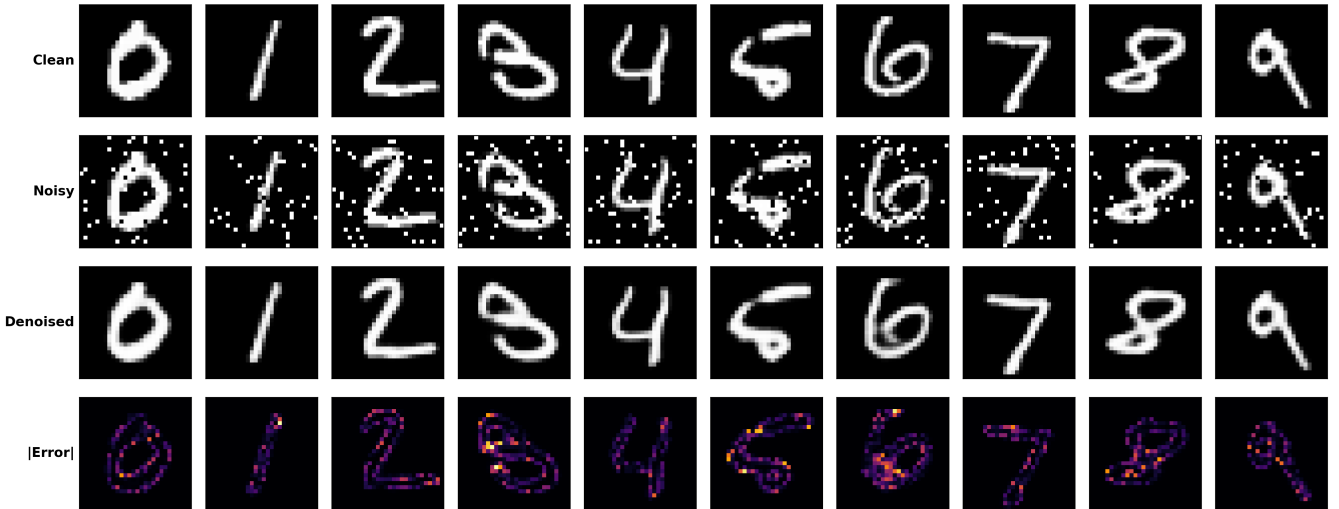


Figure 6: Plot 4(b). Salt-and-pepper noise, $p = 0.1$.

Required inference:

That an denoising CAE successfully remove Gaussian ($\sigma = 0.2$) and salt-and-pepper ($p = 0.1$) visual corruption are shown in plots 4(a) and 4(b). Rather than memorizing a random noise, this occur because a network learn to map corrupted input back to an underlying clean data manifold. Major image structures is effectively recovered and dense background noise are cleared by the model, thus.

Plot 5: Noise Level vs. Reconstruction Quality

Noise Level	MSE	MAE	SSIM
0.1	0.0020	0.0136	0.9754
0.2	0.0030	0.0170	0.9627
0.3	0.0058	0.0268	0.8700

These are the Gaussian results. Salt-and-pepper results, with the noisy input as a baseline:

Noise	Level	MSE	MAE	SSIM	Noisy input SSIM
Gaussian	0.1	0.0020	0.0136	0.9754	0.6966
Gaussian	0.2	0.0030	0.0170	0.9627	0.6125
Gaussian	0.3	0.0058	0.0268	0.8700	0.5281
Salt & pepper	0.05	0.0022	0.0137	0.9775	0.7226
Salt & pepper	0.10	0.0027	0.0153	0.9712	0.5944
Salt & pepper	0.20	0.0047	0.0202	0.9452	0.4637

Plot 5: Noise Level vs Reconstruction Quality (Gaussian)

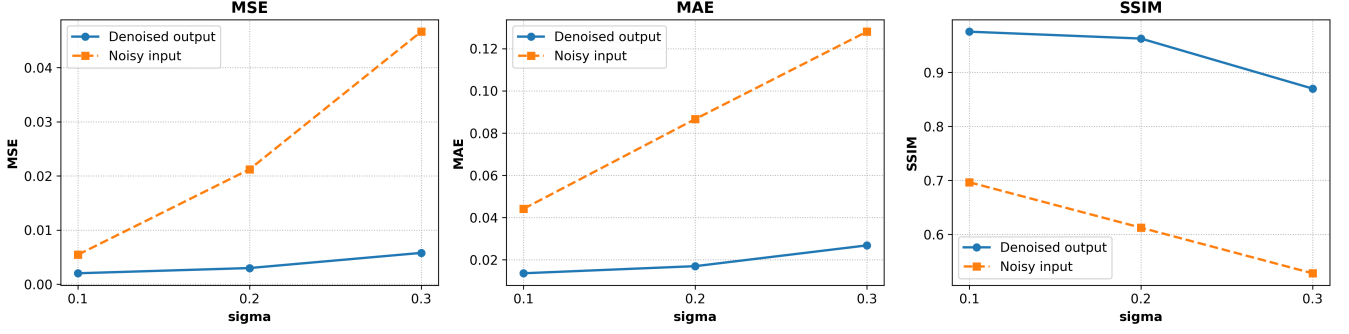


Figure 7: Plot 5(a). Gaussian: MSE, MAE and SSIM vs σ (denoised output vs noisy input).

Plot 5: Noise Level vs Reconstruction Quality (S&P)

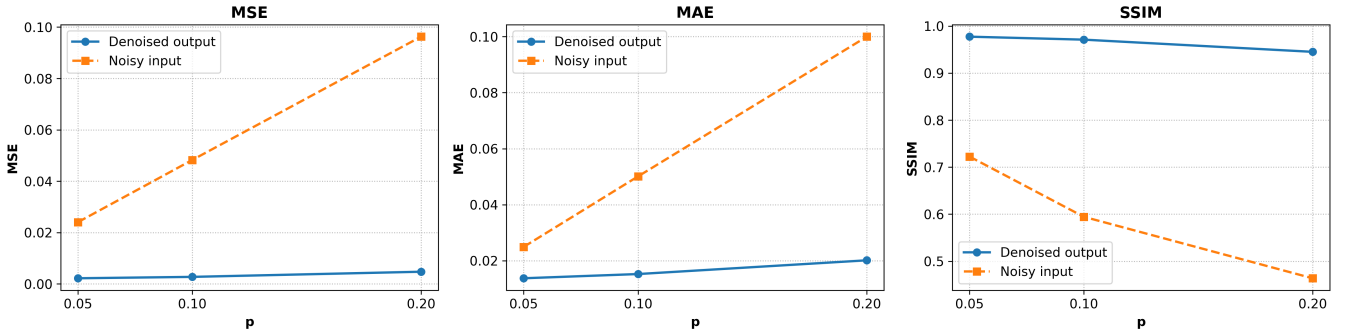


Figure 8: Plot 5(b). Salt-and-pepper: MSE, MAE and SSIM vs p .

Required inference:

Reconstruction resilience are quantified by plots 5(a) and 5(b); e.g., at $\sigma = 0.3$, a Gaussian input SSIM drop to 0.53 but an model restore it to 0.87. The denoised outputs remains significantly better than a raw inputs, while quality naturally degrade with extreme corruption that destroy essential structural feature. Across varying noise level, a highly resilient representation is ultimately learned by a denoising autoencoder.

14. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

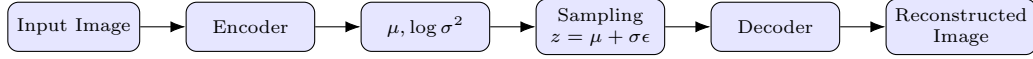
$$q_\phi(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



15. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

BCE is summed over the 784 pixels and KL is summed over the latent dimensions, both averaged over the batch. All loss values are therefore per image.

16. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

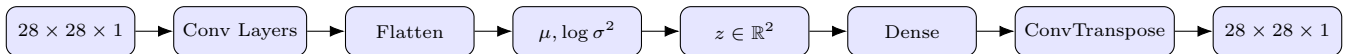
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



- Encoder: Conv(32, s2) → Conv(64, s2) → Flatten → Dense(16) → μ, log σ² (2 each) → sampling.
- Decoder: Dense(7 · 7 · 64) → Reshape → ConvT(64, s2) → ConvT(32, s2) → ConvT(1, sigmoid).
- 134,165 parameters, 30 epochs, 105.2 s.
- MSE, MAE and SSIM use the reconstruction decoded from μ(x).

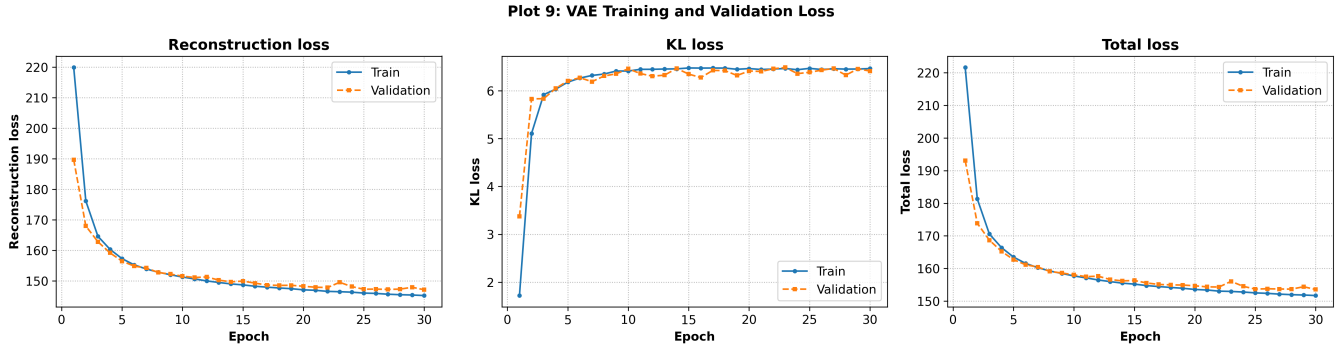


Figure 9: Plot 9. VAE reconstruction, KL and total loss (training vs validation).

Required inference:

While an KL divergence penalty rise to 6.41 before plateauing across 30 epochs, a VAE's reconstruction loss steadily drop to 146.11. Because a model prioritize reconstruction early on while a KL penalty progressively regularize a 2D latent distribution toward an standard normal $\mathcal{N}(0, I)$ prior, this occur. Balancing accurate reconstructions with probabilistic latent regularization, a VAE consequently converge smoothly.

17. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.

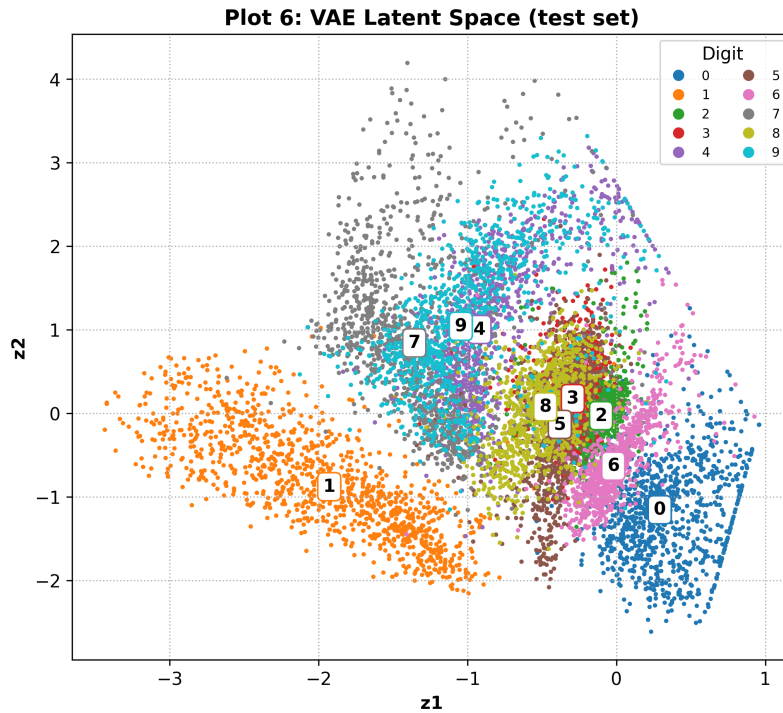


Figure 10: Plot 6. Test images encoded by the VAE (latent mean), coloured by digit.

Required inference:

Within an compact 2D latent space spanning roughly $[-3, 3]$, plot 6 visualize a 10 digit class densely clustered. Because two dimension cannot perfectly separates all classes, overlap (e.g., 4s and 9s) happen while this compactnes is driven by an KL divergence penalty enforcing a $\mathcal{N}(0, I)$ prior. Reflecting visual similarities, we concludes the VAE learn a structured, continuous latent spaces.

18. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

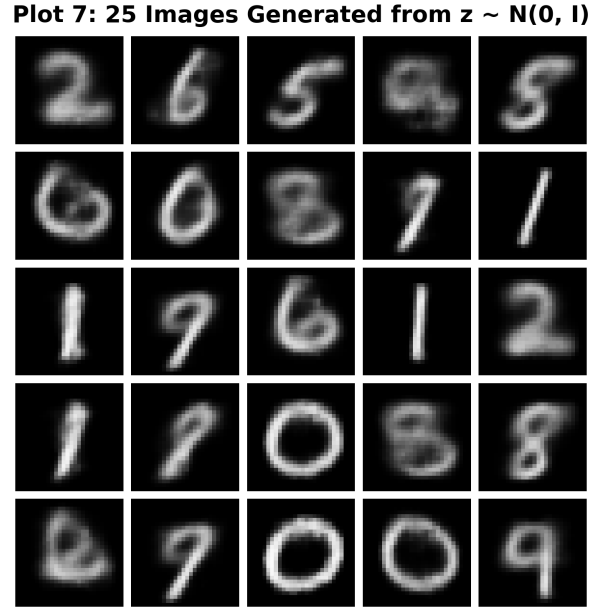


Figure 11: Plot 7. 25 images decoded from $z \sim \mathcal{N}(0, I)$.

Required inference:

By decoding random sample from an $\mathcal{N}(0, I)$ prior, plot 7 show 25 novel digits generated. While blurriness result from a BCE reconstruction loss averaging out uncertainty across overlapping cluster, this generative diversity stem from a continuous, prior-regularized latent spaces. Despite occasional blurs and class ambiguity near boundaries, novel samples is effectively generated by a VAE, thus.

19. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

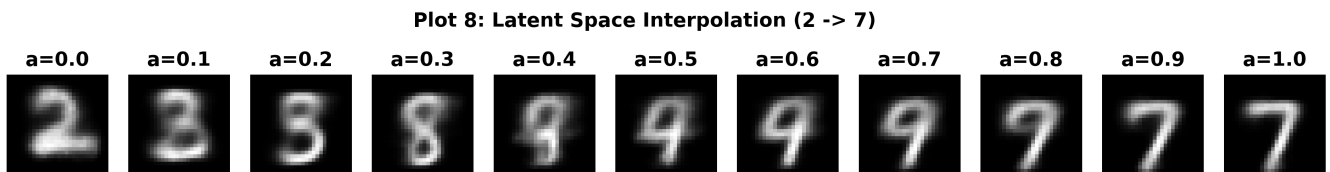


Figure 12: Plot 8. Interpolation from a test “2” (z_A) to a test “7” (z_B).

Required inference:

Between a latent vectors of an '2' and a '7', plot 8 illustrate an linear interpolation. Because an VAE's probabilistic training create a dense, continuous latent spaces without "holes," a decoded digit morph structurally through intermediate shape (like 3 and 8). Allowing natural, realistic transition between distinct data point, we concludes a VAE learn an semantically meaningful manifolds.

20. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.0133	0.0406	0.8509	211,040
Convolutional AE	0.0012	0.0102	0.9887	74,497
Denoising CAE*	0.0030	0.0170	0.9627	74,497

*Gaussian $\sigma = 0.2$ input, compared with the clean image.

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	146.11
KL Loss	6.41
Total Loss	152.52
Test MSE	0.0415
Test MAE	0.0981
Mean SSIM	0.5344

The CAE is best. The denoising CAE is close behind even though its input is noisy. The VAE is worst at reconstruction: with a 2-D latent and a KL penalty, it gives up accuracy in exchange for being able to generate new images.

21. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.

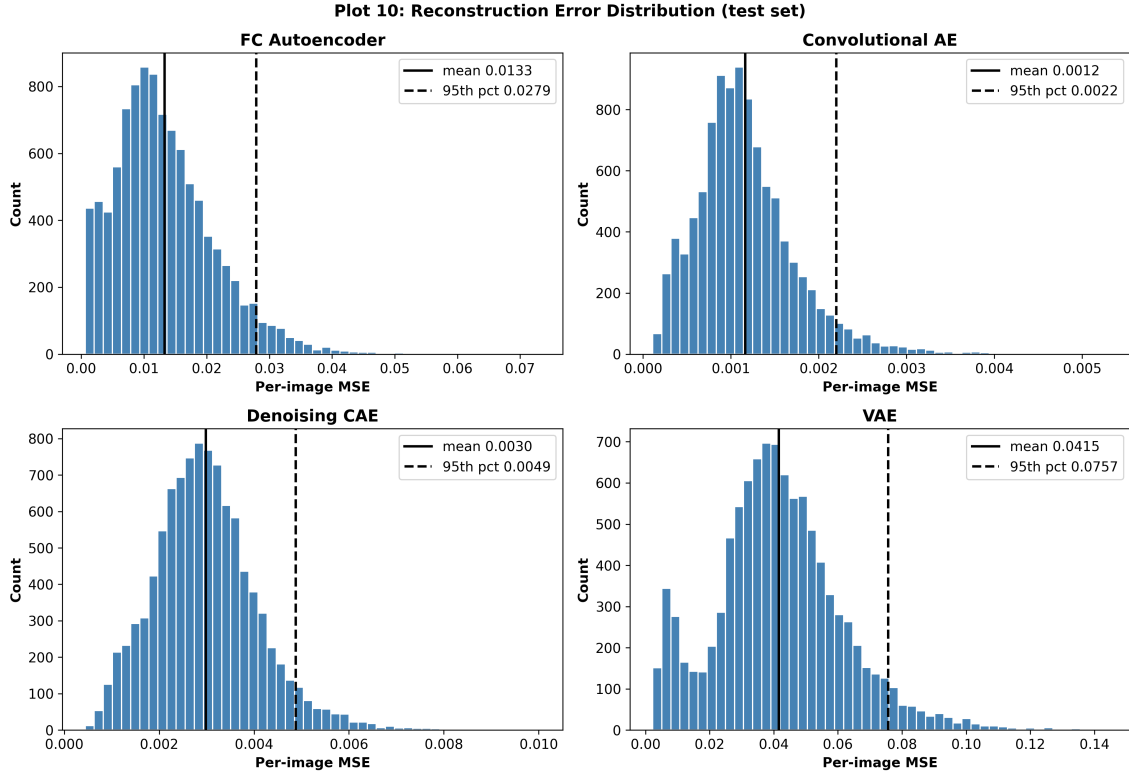


Figure 13: Per-image reconstruction error for all four models (solid: mean, dashed: 95th percentile).

Model	Mean	Median	95th pct	Max	Corr(MSE, SSIM)
FC Autoencoder	0.0133	0.0120	0.0279	0.0732	-0.88
Convolutional AE	0.0012	0.0011	0.0022	0.0053	-0.78
Denoising CAE	0.0030	0.0029	0.0049	0.0100	-0.57
VAE	0.0415	0.0404	0.0757	0.1441	-0.82

Required inference:

With an long tail of outlier extending up to 0.0732, plot 10 reveal highly concentrated low reconstruction error (e.g., FC-AE mean MSE 0.0133). While an long tail comprise rare, irregular handwriting style a bottleneck struggle to encodes, this occur because most digit matches easily compressible common pattern. While irregular outlier remains challenging, we concludes model confidently reconstructs typical digit.

22. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Sample	Test index	Digit	MSE	SSIM
1	1782	8	0.0732	0.360
2	1253	4	0.0677	0.450
3	4601	8	0.0542	0.558
4	3846	6	0.0534	0.489
5	3778	5	0.0517	0.505

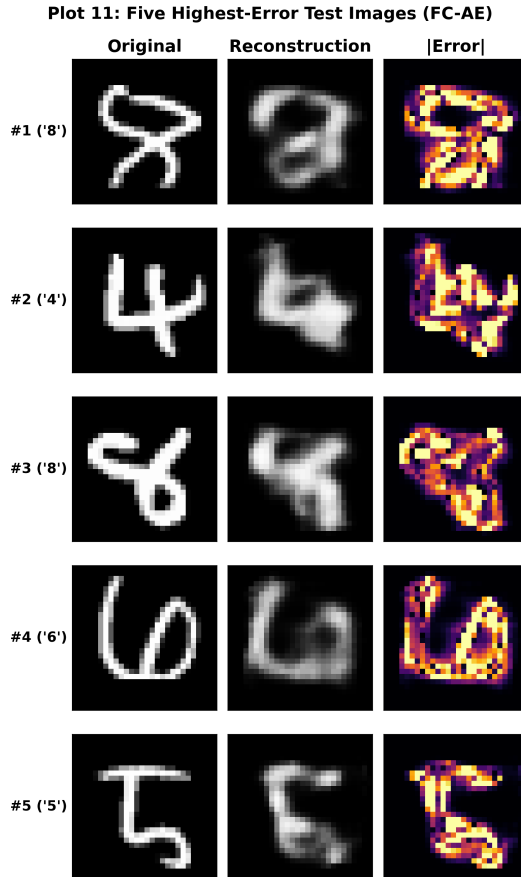


Figure 14: Five highest-error test images (FC-AE): original, reconstruction and error.

Why hard: of their digit, all five is unusual version. Very thick stroke or open, crossed loop are had by an 8s, a 4 have an odd shapes, a 6 is very wide, and an 5 look like a J. So these gets reconstructed as blurry “average” digit, a 16-D code only capture common shape.

23. Source Code

GitHub repository: https://github.com/the-future-codemaster/dl_lab/tree/main/Experiment_7

24. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time (s)
FC Autoencoder	0.0133	0.0406	0.8509	211,040	39.3
Conv. Autoencoder	0.0012	0.0102	0.9887	74,497	68.9
Denoising CAE	0.0030	0.0170	0.9627	74,497	116.7
VAE	0.0415	0.0981	0.5344	134,165	105.2

Time is the training time. The Denoising CAE row uses Gaussian $\sigma = 0.2$ inputs.

25. Required Inferences

Differences between deterministic and probabilistic latent representations:

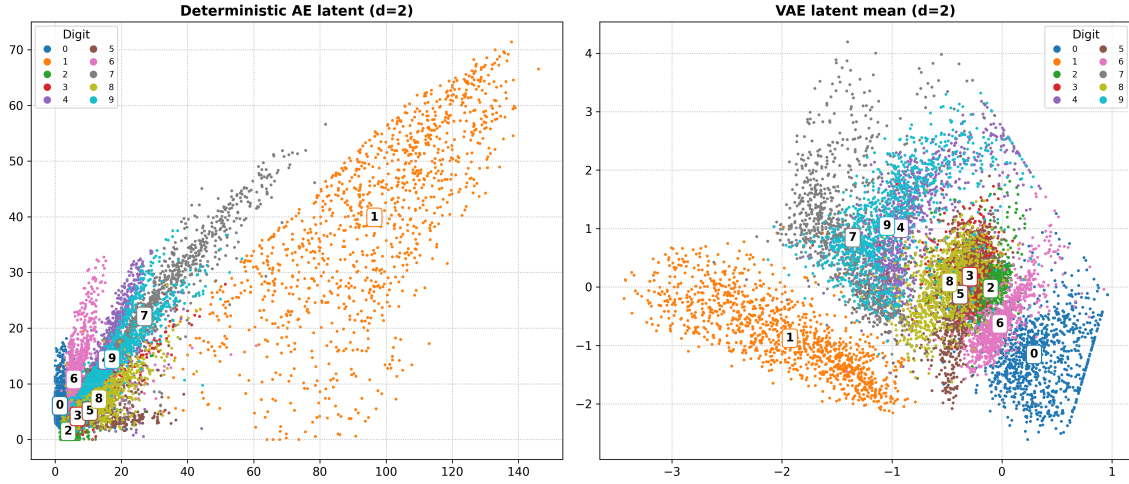


Figure 15: Latent space of the deterministic FC-AE ($d_z = 2$) vs the VAE ($d_z = 2$).

Meaning each inputs map to an single point, the AE latent space are deterministic. Because of a ReLU latent layer, it is unbounded (up to about 140) and only positive. Large gap between cluster are also had by it. To meaningful digits, random point sampled in that space does not decodes. Mapping input to distribution, a VAE latent space is probabilistic. Matching an prior $\mathcal{N}(0, I)$, it is regularized by a KL divergence to be compact, continuous, and centered around a origin (roughly ± 3). To generates valid, realistic digit, this smooth, densely packed structures allow sampling $z \sim \mathcal{N}(0, I)$. To functions as an powerful generative models, this fundamental difference is what enable a VAE.

26. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Latent Dimension	MSE	SSIM	Parameters
2	0.0410	0.5464	210,130
8	0.0197	0.7817	210,520
16	0.0133	0.8509	211,040
32	0.0100	0.8890	212,080

Plot 10: Latent Dimension vs. Reconstruction MSE

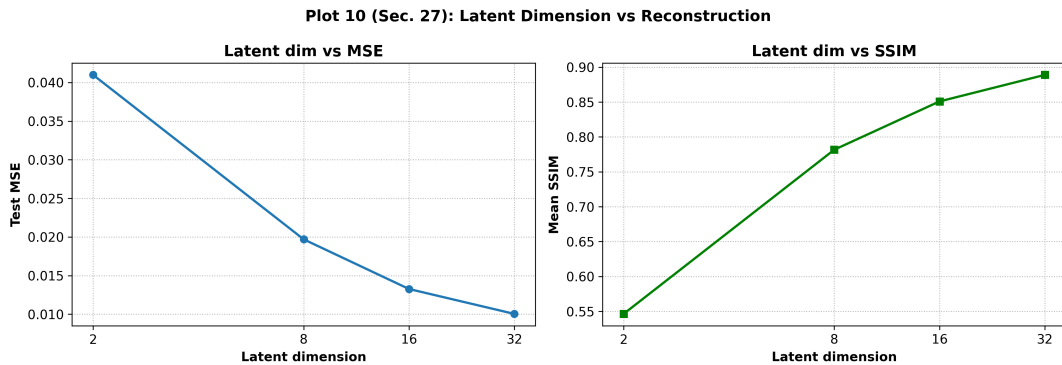


Figure 16: Plot 10. Latent dimension vs MSE and SSIM.

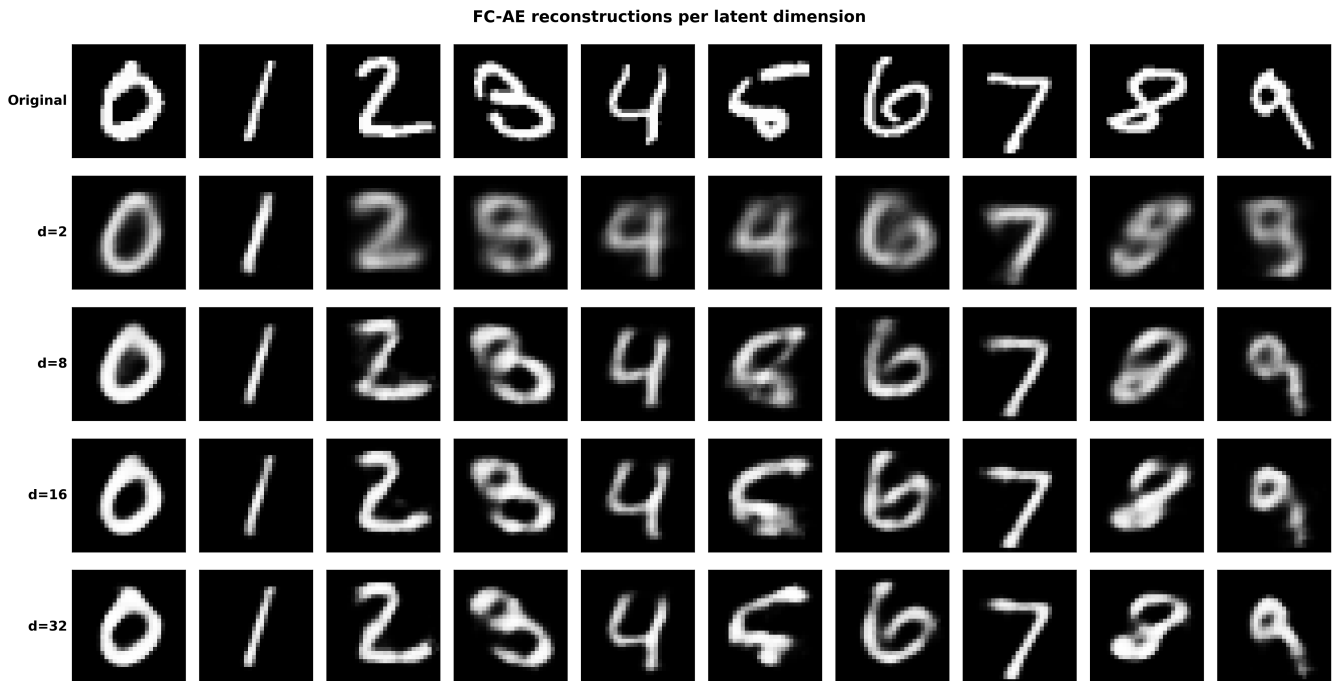


Figure 17: FC-AE reconstructions for each latent dimension.

Required inference:

With diminishing return beyond that, plot 11 show FC-AE reconstruction MSE dropping significantly from 0.0410 at $d_z = 2$ to 0.0133 at $d_z = 16$. While larger dimension eventually captures a dataset's intrinsic dimensionality, this happen because an 2-D bottleneck force severe information loss. With $d_z = 16$ offering a optimal balances for this architectures, there is an fundamental compression-fidelity tradeoff consequently.

27. Discussion Questions

1. What is an autoencoder?

Designed to learns efficient data coding, a autoencoder are an unsupervised artificial neural networks. It is trained to reconstruct its own input at the output layer by passing the data through a lower-dimensional bottleneck. Filtering out noises and capturing essential feature without requiring labeled datas, this force a network to learns an compressed, meaningful representation (latent spaces) of a data.

2. What is the purpose of the encoder?

To acts as an feature extractors and compressor are a encoder's roles. It takes the high-dimensional input data (e.g., a 28×28 image) and maps it through hidden layers into a lower-dimensional space, generating a compact latent vector that retains the most critical information required to recreate the input.

3. What is the purpose of the decoder?

Taking an compressed latent vectors and reconstructing it back into an original high-dimensional input spaces, a decoder serve as an generative mappings. It interprets the encoded features to rebuild the spatial structure and pixel intensities as accurately as possible.

4. What is a latent representation?

Produced by an encoder, a latent representations, or latent codes, are a compressed, continuous, lower-dimensional vectors. It represents the most salient underlying features, patterns, and variations of the data, serving as the information bottleneck between the encoder and decoder.

5. Why is the latent representation usually smaller than the input?

From simply learning a identity functions (memorizing and directly passing input to output), creating a information bottleneck prevent an network. By forcing the data through a smaller dimension, the network must learn robust, generalized features and discard redundant information or noise.

6. What happens when the latent dimension is too small?

An network suffer from severe informations loss (underfitting), if a latent dimensions are too small (e.g., $d_z = 2$). It lacks the capacity to capture enough distinguishing features, causing reconstructions to be heavily blurred, lack fine details, or even morph into entirely incorrect classes (e.g., a 5 reconstructed as a 4).

7. What happens when the latent dimension is increased?

Leading to sharper and more accurate reconstruction, increasing an latent dimensions increase a model's capacities to stores informations. However, if it becomes too large, it risks learning an identity mapping, storing noise, and failing to extract a meaningful compressed representation, reducing its utility for dimensionality reduction.

8. **Why can a convolutional autoencoder preserve spatial information more effectively?**
Using localized filter and weight sharings, convolutional layer are unlike fully connected layer that flattens an input and destroys a 2D spatial arrangements. This allows them to preserve the spatial hierarchy of pixels, effectively capturing local patterns such as edges, corners, and strokes, which are critical for image structures.
9. **Differentiate an autoencoder and a denoising autoencoder.**
Learning a data distributions, an standard autoencoder take an clean inputs and learn to reconstructs it. A denoising autoencoder takes an artificially corrupted or noisy input and is trained to reconstruct the original clean image. This forces the model to learn a more robust representation that distinguishes actual data features from random noise.
10. **Why is the clean image used as the target in denoising?**
Providing a supervised signals necessary for an model to learns a mapping from an noisy manifolds back to a clean data manifolds, the clean images are used as a targets. If the noisy image were the target, the model would simply learn to replicate the noise, defeating the purpose of denoising.
11. **What happens when the noise level is increased?**
Making it harder for an model to recovers an original images, a structural integrity of a input degrade as a corruption level increase. While the model remains robust up to its trained noise level, extreme noise will obscure essential features, leading to loss of fine details and lower reconstruction quality (e.g., lower SSIM).
12. **Why are MSE, MAE and SSIM useful for image reconstruction?**
Making it good for overall pixel intensity matchings, MSE penalize large outlier error aggressively. MAE treats all errors equally, providing a robust average pixel difference. SSIM (Structural Similarity Index) evaluates perceptual quality by measuring changes in luminance, contrast, and structure, aligning much closer to human visual perception than pixel-wise metrics.
13. **What is a Variational Autoencoder?**
Instead of mapping input to an fixed vectors, an VAE are a probabilistic generative models that map them to an probability distributions in a latent spaces. By sampling from this distribution and regularizing it to match a prior (usually a standard normal distribution), the VAE can generate novel, realistic data samples.
14. **How does a VAE differ from a conventional autoencoder?**
Mapping a inputs to an single discrete points in a latent spaces, an conventional autoencoder are deterministic. A VAE is probabilistic; its encoder outputs parameters (mean and variance) defining a distribution. It uses the reparameterization trick for sampling during training and incorporates a KL divergence term in the loss function to regularize the latent space.
15. **Why does a VAE learn a distribution instead of a single latent vector?**
Ensuring a latent spaces are continuous, densely populated, and smooths, learning an distributions introduce structured uncertainties. This prevents "holes" in the latent space; if a point is sampled between two encoded distributions, it will still decode into a meaningful and realistic output, enabling generation and smooth interpolation.
16. **What is the reparameterization trick?**
Detaching a randomness, a reparameterization trick allow backpropagations through an random sampling processes. Instead of sampling z directly from $\mathcal{N}(\mu, \sigma^2)$, z is expressed as $z = \mu + \sigma \odot \epsilon$, where ϵ is sampled from $\mathcal{N}(0, I)$. This makes the sampling operation differentiable with respect to μ and σ .
17. **Why is ϵ sampled from a standard normal distribution?**
Providing an stable, parameter-free sources of stochasticity, sampling ϵ from an standard normal distributions $\mathcal{N}(0, I)$ are. It scales easily with the learned variance and shifts with the learned mean, simplifying the back-propagation process while keeping the network weights independent of the random noise generation.
18. **What is the role of KL divergence in the VAE loss?**
Forcing an learned latent distribution (a posterior) to be as close as possibles to a prior distributions (typically $\mathcal{N}(0, I)$), a KL divergence terms act as an regularizers. This restricts the distributions from spreading infinitely apart or shrinking to zero variance, ensuring a compact, continuous, and structured latent space suitable for generation.
19. **What is the prior distribution used in the VAE?**
Meaning it have an mean of zero and a variances of one across all dimension, an standard prior are a isotropic standard normal distributions, $\mathcal{N}(0, I)$.
20. **Why is a two-dimensional latent space useful for visualization?**
To be plotted directly onto an standard 2D scatter plots, an 2D latent spaces allow a exact encoded coordinate of a datas. This provides immediate, interpretable visual insight into how the network clusters different classes and structures the latent manifold, without requiring dimensionality reduction techniques like PCA or t-SNE, which can introduce distortions.
21. **What does latent-space interpolation demonstrate?**
A smoothness and continuities of an learned latent manifolds are demonstrated by interpolations. By decoding points along a linear path between two latent vectors, we can observe whether the generated images transition smoothly and semantically from one class to another, proving the model has learned a meaningful structural continuum rather than disjoint memorized regions.

22. **Why can a VAE generate new images?**

A latent spaces are densely packed and continuous, because a KL divergence regularizers force an aggregated latent distributions to matches an standard normal priors. This makes random sampling from the prior more likely to produce plausible samples from the learned distribution.

23. **Why might some VAE-generated images be unclear?**

Encouraging a model to predicts an "average" of possible variation to minimizes expected errors, blurrines often arise because an reconstruction loss (like MSE or BCE) penalize pixel-wise difference. Additionally, if a sampled point lies on the boundary between two digit clusters in the latent space, the decoder produces an ambiguous interpolation of both shapes.

24. **What is the difference between reconstruction and generation?**

Taking an specific existing inputs, encoding it into a latent spaces, and decoding it back to recovers an original datas are a process of reconstructions. Generation involves sampling a completely new, random point from the latent prior distribution (without any original input) and decoding it to synthesize a novel data sample that did not previously exist.

25. **Why should test images remain unseen during training?**

Rather than its abilities to simply memorizes a training set, keeping test image strictly unseen ensure that a evaluation metric measures an model's true ability to generalizes to novel datas. This is critical for assessing real-world performance, overfitting, and the robustness of the learned representations.

28. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.
2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

Exercise 1: CAE Latent Dimension

A dense bottleneck of size d was added between the conv encoder and decoder.

d	MSE	MAE	SSIM	Parameters	FC-AE MSE
4	0.0286	0.0719	0.6937	102,725	—
8	0.0152	0.0442	0.8316	127,817	0.0197
16	0.0069	0.0258	0.9282	178,001	0.0133
32	0.0034	0.0173	0.9638	278,369	0.0100

Exercise 1: CAE reconstructions vs latent dimension

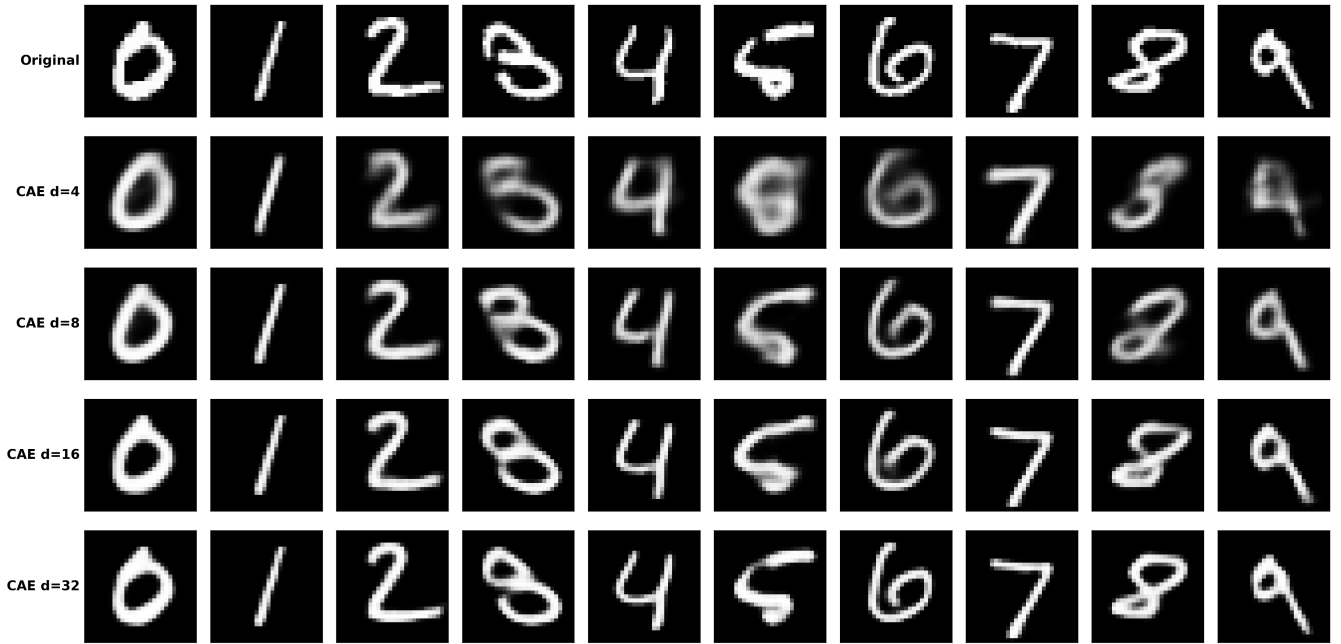


Figure 18: Exercise 1. CAE reconstructions for $d = 4, 8, 16, 32$.

Quality rises steadily with d : $d = 4$ blurs 3, 5, 8 and 9, while $d = 32$ is nearly exact. At equal d the CAE beats the FC-AE, with about $2\times$ lower MSE at 16 and $3\times$ at 32. So convolution helps on its own, not just through a bigger latent.

Exercise 2: Gaussian vs Salt-and-Pepper

SSIM (rows: training noise; columns: test noise):

	G 0.1	G 0.2	G 0.3	S&P 0.05	S&P 0.1	S&P 0.2
Trained on Gaussian 0.2	0.975	0.963	0.870	0.887	0.773	0.625
Trained on S&P 0.1	0.613	0.561	0.516	0.977	0.971	0.945
Noisy input	0.697	0.613	0.528	0.723	0.594	0.464

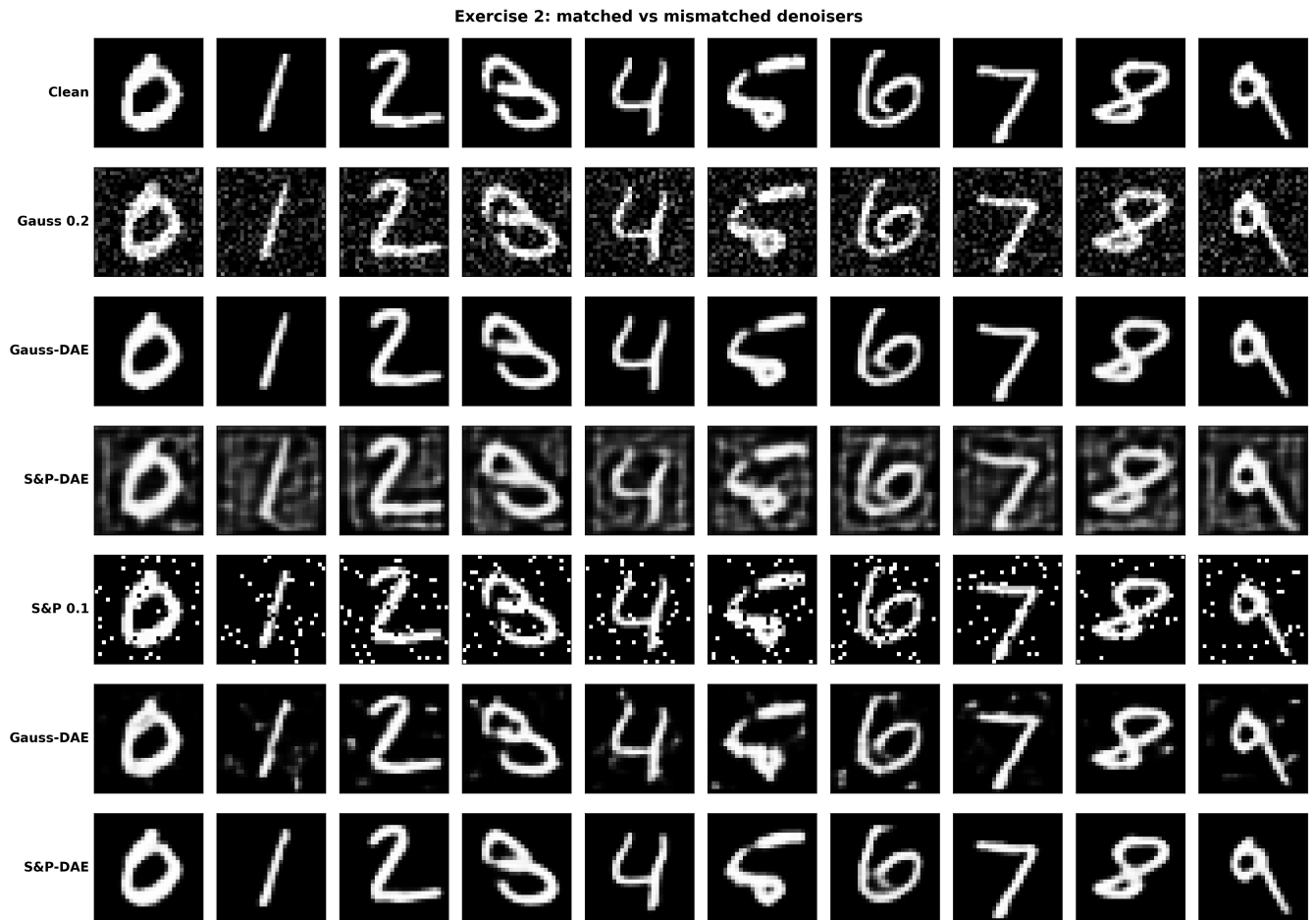


Figure 19: Exercise 2. Each noise type cleaned by the matched and the mismatched denoiser.

On a noise it was trained on, each models only work. On salt-and-pepper image, an Gaussian models leave white speck. Doing worse than doing nothings (SSIM 0.61 vs 0.70) and adding an grid-like patterns is a salt-and-pepper model on Gaussian noises. When matched, salt-and-pepper are easier to removes: even at $p = 0.2$, SSIM stay at 0.945.

Exercise 3: Two Training Noise Levels

Test σ	SSIM (trained 0.1)	SSIM (trained 0.3)	Noisy input
0.1	0.979	0.968	0.697
0.2	0.815	0.959	0.613
0.3	0.660	0.941	0.528
0.4	0.593	0.867	0.446
0.5	0.531	0.708	0.375

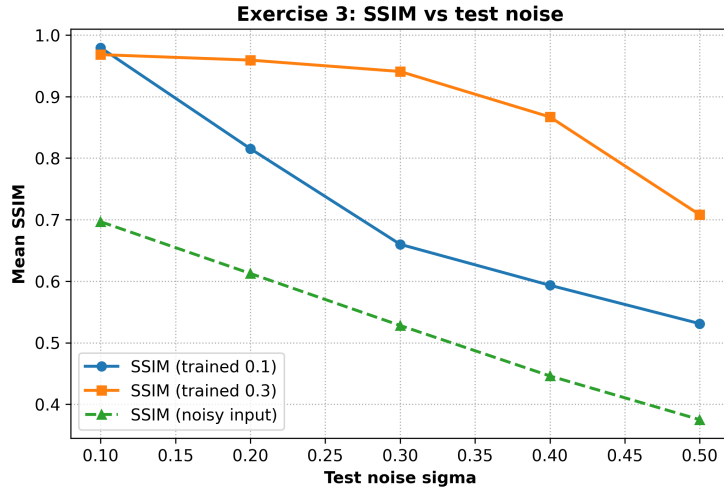


Figure 20: Exercise 3. SSIM vs test noise for models trained at $\sigma = 0.1$ and 0.3 .

Only at 0.1, an $\sigma = 0.1$ models is slightly better, and fall off quickly after that. Robust up to 0.3 and still degrading gracefully beyond it are a $\sigma = 0.3$ model. Generalising better, training with more noises are.

Exercise 4: UpSampling vs Conv2DTranspose

Decoder	MSE	MAE	SSIM	Parameters	Time (s)
UpSampling2D	0.0012	0.0102	0.9887	74,497	68.9
Conv2DTranspose	0.0009	0.0082	0.9924	74,497	78.9

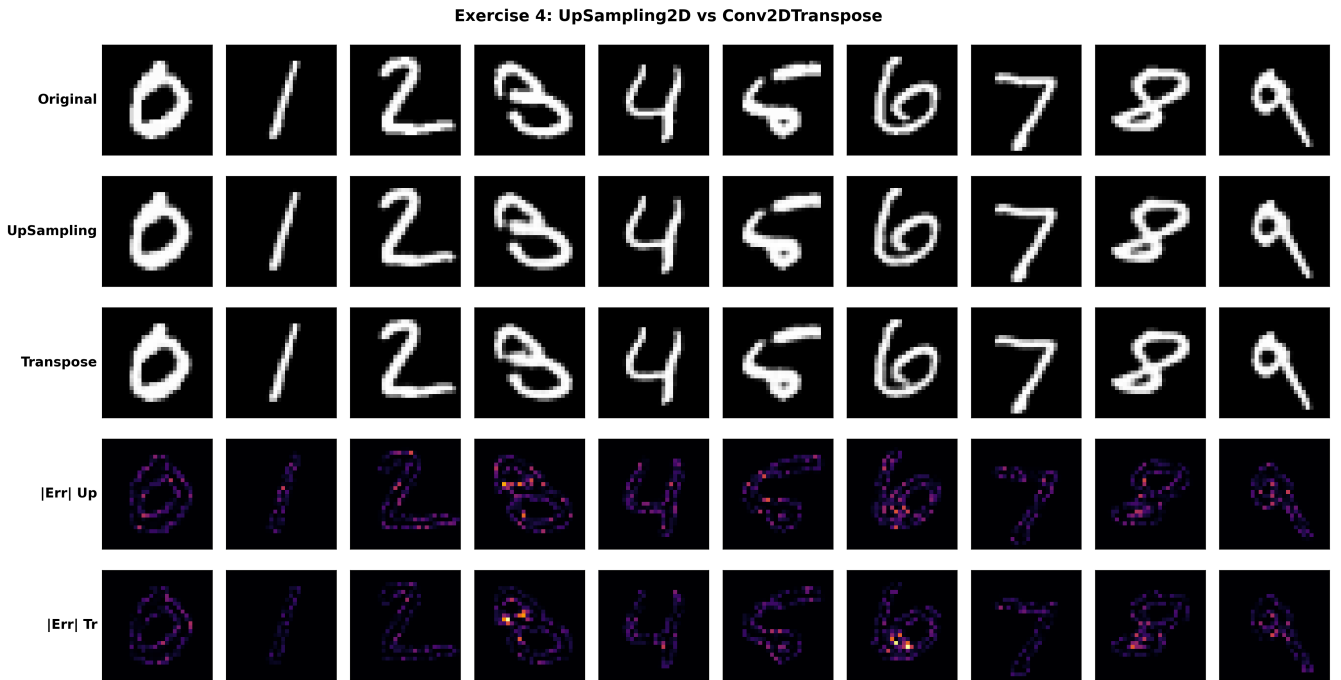


Figure 21: Exercise 4. Reconstructions and error maps for both decoders.

An same parameter counts is had by a two. Since it learn a upsampling instead of copying pixel, transposed convolutions are slightly better. Only an few brighter error pixel on a thick stroke of a 3 and 6 is visible, no clear checkerboard artifact are visible.

Exercise 5: VAE with $d_z = 2$ vs $d_z = 8$

d_z	Recon loss	KL loss	MSE	SSIM	5-NN acc.
2	146.11	6.41	0.0415	0.534	0.684
8	130.65	8.43	0.0346	0.628	0.747

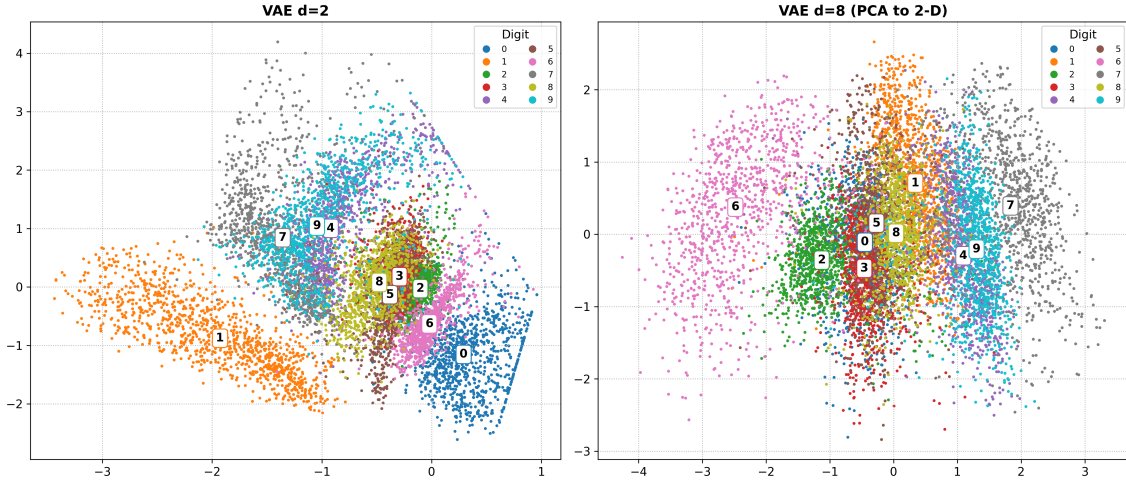


Figure 22: Exercise 5. Latent space: $d_z = 2$ (raw) and $d_z = 8$ (PCA to 2-D).

At an higher KL costs, $d_z = 8$ reconstruct better and separate a class better (5-NN accuracy 0.75 vs 0.68). Because projecting 8 dimension down to 2 hide an separation, its PCA plots look more mixed than a raw 2-D plots. More varied are sample from $d_z = 8$.

Exercise 6: Increasing the KL Weight (β)

β	Recon loss	KL loss	MSE	SSIM	5-NN acc.
1	146.11	6.41	0.0415	0.534	0.684
2	146.26	5.27	0.0413	0.543	0.655
4	150.01	4.04	0.0417	0.524	0.687
8	155.10	3.13	0.0427	0.503	0.671

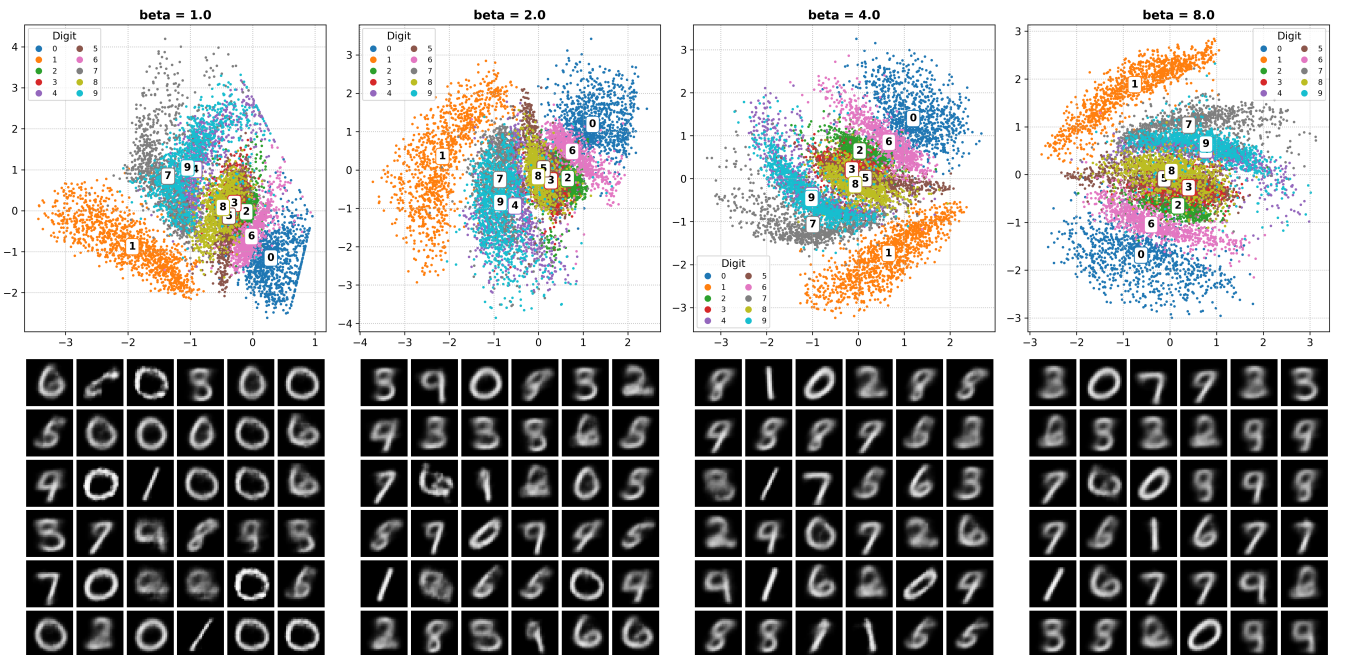


Figure 23: Exercise 6. Latent space (top) and random samples (bottom) for $\beta = 1, 2, 4, 8$.

A higher β steadily lowers KL ($6.4 \rightarrow 3.1$) and raises reconstruction loss ($146 \rightarrow 155$). SSIM drops slightly at $\beta = 8$. The latent spreads out into a rounder, more prior-like shape, while the samples look much the same. The 5-NN accuracy barely changes. It's a trade-off: a more regular latent space costs reconstruction quality.

Exercise 7: Larger Sample Grid and Diversity

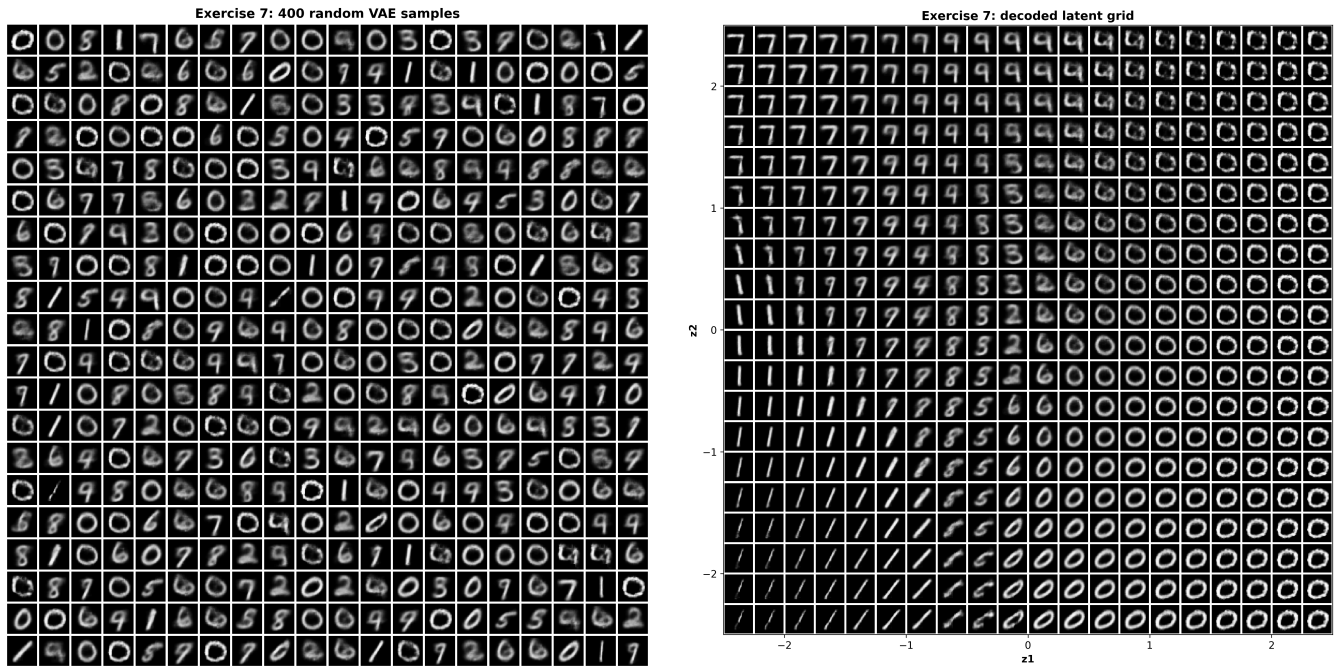


Figure 24: Exercise 7. 400 random samples (left) and a decoded grid over the latent space (right).

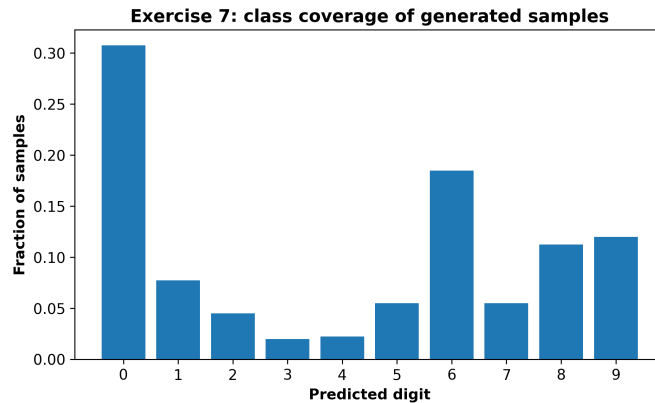


Figure 25: Exercise 7. Predicted class of the 400 samples (small CNN classifier).

Class entropy (max 3.32 bits)	2.88
Mean classifier confidence	0.856
Ambiguous samples (confidence < 0.6)	15.5%

Not evenly but all ten digit appears: while 3 and 4 is only about 2% each, 0 are 31% and 6 are 18%. Why is shown by a manifolds: while 3, 5 and 8 shares an thin bands and 4 are merged into 9, 0 and 6 takes up large area. From an boundaries between this regions, about 15% of sample is ambiguous blend.

Exercise 8: Interpolation Between Digit Regions

Paths run between the median latent codes of two digits.

Exercise 8: region-to-region interpolation (title = predicted digit, confidence)

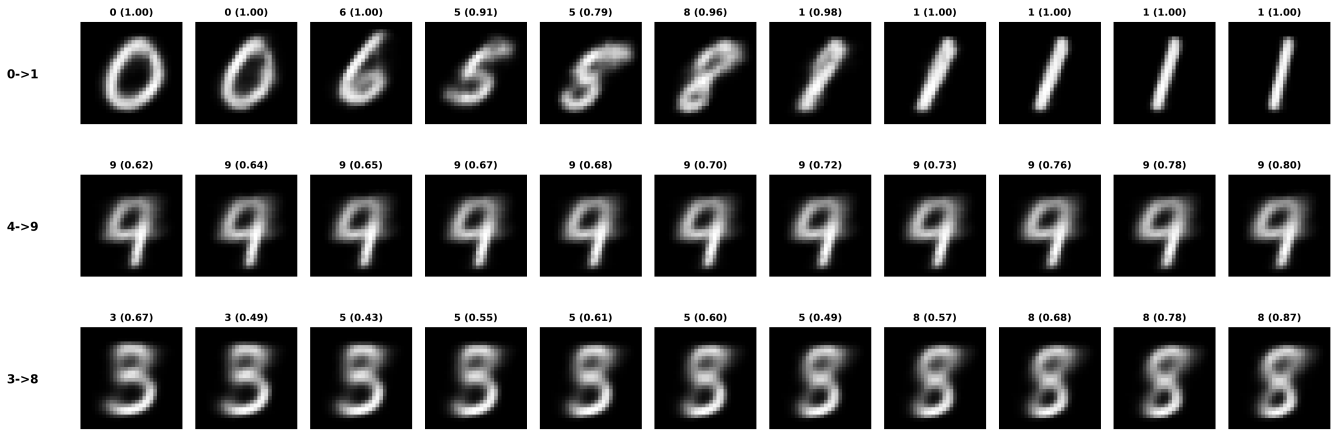


Figure 26: Exercise 8. Decoded paths 0→1, 4→9, 3→8 (title: predicted digit, confidence).

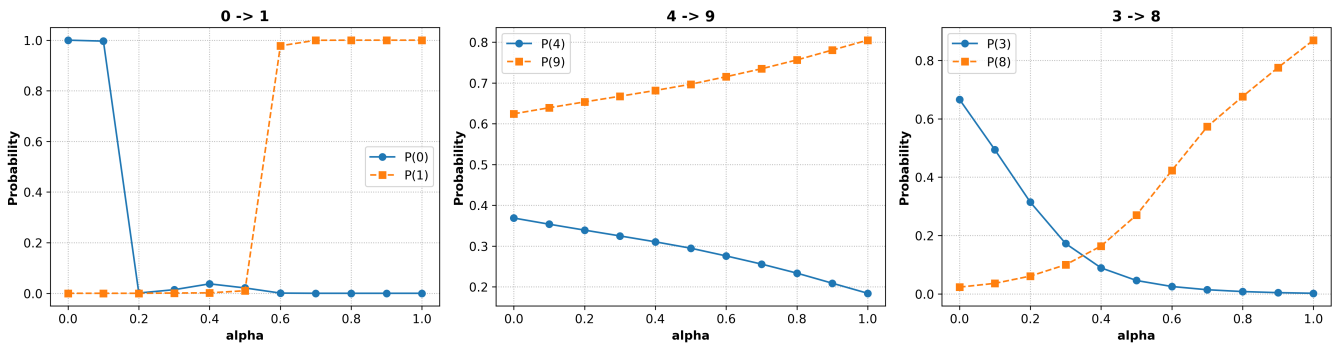


Figure 27: Exercise 8. Classifier probability of the two end digits along each path.

- 0 → 1: P(0) drops to about 0 by $\alpha = 0.2$, and P(1) only takes over from $\alpha = 0.6$. In between, the path passes through 6, 5 and 8.
- 4 → 9: every frame is classified as 9, and P(9) rises from 0.62 to 0.80. The 4 and 9 regions overlap, so even the “4” centre decodes as 9-like.
- 3 → 8: a smooth crossover at $\alpha \approx 0.4$, passing through 5.

So interpolation is smooth, but a clean A → B morph only happens when the two regions are adjacent and well separated.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image → Encoder → Latent Representation → Decoder → Reconstructed Image

They should also demonstrate denoising:

Clean Image → Noise → Denoising Autoencoder → Recovered Image

and variational generation:

Image → (μ, σ) → z → Decoder → Generated/Reconstructed Image

All three pipelines were implemented. The CAE reconstructs best (SSIM 0.989), the denoising CAE recovers clean digits from noisy inputs, and the VAE trades reconstruction accuracy (SSIM 0.534) for a smooth latent space that can generate new digits.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, “Auto-Encoding Variational Bayes,” ICLR, 2014.
3. Pascal Vincent et al., “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion,” Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, “MNIST Handwritten Digit Database.”
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>